Sparse-LD and BED Workflows

Peter Sørensen

Table of contents

This note describes the practical bridge from individual-level PLINK BED genotypes and aligned phenotypes to a disk-backed sparse-LD representation and the sufficient statistics required by `stblr_csr()`. The same starting data can also be fitted directly with `stblr_bed()`, making the workflow useful for understanding and checking the relationship between the individual-level and summary-statistics paths.

The examples below are intentionally short. The complete executable sequence is in the sparse-LD/BED workflow. Its fitting steps are computationally expensive and its MCMC settings are demonstration settings rather than recommendations for real analysis.

1 One Genotype Source, Two Fitting Paths

The workflow begins with PLINK BED files, marker metadata, and one or more phenotypes. It then branches into two model-input representations:

```
BED files + phenotype y
        |
        v
align markers with cls and select sample rows
        |
        +--> make_stats()
        |       |
        |       v
        |    stats$wy, stats$ww, stats$yy, n, marker names
        |
        +--> make_sparseLD()
                |
                v
             Glist$sparseLD$prefix
                |
                +--> sparseLD_read_CSR() for inspection
                |
                v
             stblr_csr()
                |
                v
          CSR posterior summaries

BED files + phenotype y + the same cls/rows/scaling
        |
        v
stblr_bed()
        |
        v
optional BED-versus-CSR posterior comparison
```

The CSR path converts individual-level data into reusable cross-products and sparse LD files. The direct BED path retains access to observed genotypes during sampling. Both paths represent related marker-update calculations, but they are not numerically identical by construction.

The statistical role of each representation is developed in Data representations. This note focuses on object handoffs and practical contracts.

2 Selecting Samples and Markers From a BED-Backed `Glist`

A qgg `Glist` connects biological marker identities to BED file columns and allele frequencies. The principal objects are:

Object	Practical role
`Glist$bedfiles`	Paths to one or more PLINK BED files containing individual-level genotypes.
`n`	Number of individuals encoded in the BED files.
`Glist$rsids[[chr]]`	Marker identities in BED column order for a chromosome.
`Glist$rsidsLD[[chr]]`	Selected marker identities in the intended LD/model order.
`cls`	BED column indices that map the intended LD/model order into `Glist$rsids`.
`rows`	Optional 1-based BED row indices selecting individuals.
`af` or `maf`	Allele frequencies or MAF corresponding exactly to `cls`, used for genotype scaling and MAF-dependent priors.
`y`	Phenotype vector or matrix aligned to all individuals or to `rows`.

The workflow establishes marker order with:

```
chr <- 1
marker_id <- Glist$rsidsLD[[chr]]
cls <- match(marker_id, Glist$rsids[[chr]])
stopifnot(!anyNA(cls))

bed_files <- Glist$bedfiles[chr]
af <- Glist$af[[chr]][cls]
```

`cls[j]` identifies the BED column containing `marker_id[j]`. Its order, not just its set of values, determines the order of every downstream marker-indexed object.

For one BED file, `make_stats()` and `make_sparseLD()` can infer `cls` from `match(Glist$rsidsLD[[chr]], Glist$rsids[[chr]])` when it is not supplied. For multiple BED files, each `cls[[k]]` selects and orders markers from `Glist$bedfiles[chr[k]]`; the concatenated selection is the model order.

`n` is the total number of BED rows. If `rows = NULL`, all BED rows are used and `y` should have `n` rows. If `rows` is supplied, it selects the individuals used to construct statistics and LD, and `y` must describe that same selected sample in the expected order. The same `rows` value must be used for both branches of CSR preparation.

3 Simulating or Supplying Aligned Phenotypes

The workflow example simulates traits for `Glist$rsidsLD[[chr]]` and then scales the resulting phenotype matrix:

```r
sim <- mtsim(Glist = Glist, chr = chr, rsids = marker_id, nt = 3, seed = 1)
y <- as.matrix(scale(sim$y))
```

Real analyses can replace this with observed phenotypes, but alignment and preprocessing must be explicit:

- rows of `y` must correspond to BED individuals or the selected `rows`;
- columns of `y` define traits and should have stable names;
- centering and scaling choices affect `stats$yy` and prior initialization;
- missing phenotype handling must be consistent with the chosen sample.

For trait t, `make_stats()` constructs quantities including

$$wy_t = X^\top y_t, \qquad ww_t = \mathrm{diag}(X^\top X), \qquad yy_t = y_t^\top y_t.$$

The sparse-LD branch must use the same version of X, including sample rows and marker scaling, that generated these statistics.

4 Path A: Direct Individual-Level Fitting With `stblr_bed()`

Use `stblr_bed()` when individual-level BED genotypes and aligned phenotypes remain available during fitting and a separate sparse-LD artifact is not required. The sampler reads selected genotype blocks from BED and uses observed marker values directly.

```r
fit_bed <- stblr_bed(
  Glist = Glist,
  y = y,
  method = "bayesC",
  chr = chr,
  cls = cls,
  scale = TRUE,
  rows = NULL,
  pi_init = 0.001,
  h2 = 0.5,
  nit = 1000,
  nburn = 100,
  seed = 10
)
```

This path is appropriate when:

- individual-level data access is allowed throughout analysis;
- reading BED blocks during sampling is acceptable;
- observed genotype information is preferred to a thresholded LD approximation;
- the fit is being used as a reference for CSR preparation or model behavior.

`stblr_bed()` derives BED paths, allele frequencies, and marker names from `Glist`. If `cls` is omitted, the wrapper matches `Glist$rsidsLD` against `Glist$rsids`; supplying `cls` explicitly makes the comparison contract visible.

5 Path B, Step 1: Deriving `wy`, `ww`, and `yy` With `make_stats()`

`make_stats()` converts the selected BED genotypes and phenotype matrix into sufficient statistics for ST-BLR fitting:

```
stats <- make_stats(
  Glist = Glist,
  y = y,
  chr = chr,
  cls = cls,
  rows = NULL,
  scale = TRUE,
  nthreads = 4
)
```

The main output handoff is:

Object	Meaning and use
`stats$wy`	Marker-trait cross-products $X^\top Y$; drives marker-residual scores.
`stats$ww`	Per-marker diagonal genotype cross-products; used by marker updates.
`stats$yy`	Per-trait phenotype sums of squares $y_t^\top y_t$; used for likelihood and variance initialization.
`stats$n`	Sample size when returned; otherwise supply compatible `n` to `stblr_csr()`.
`stats$m`	Marker count when returned; otherwise inferred from `stats$ww[[1]]`.

`stats$wy` and `stats$ww` must each describe the same m markers, in the same order later used by the CSR LD matrix. `stats$yy` has one entry per trait, not one entry per marker.

`make_stats()` accepts a single chromosome/file or a list of chromosomes/files through `chr` and `cls`, and records the resolved marker order in `stats$marker_names`.

6 Path B, Step 2: Computing Sparse LD With `make_sparseLD()`

`make_sparseLD()` computes marker correlations from BED-backed `Glist` data, applies distance and r^2 retention rules, writes a disk-backed CSR representation, and stores the resolved sparse-LD metadata in `Glist$sparseLD`:

```
Glist <- make_sparseLD(
  Glist = Glist,
  out_prefix = file.path(output_dir, "chr1_ld"),
  chr = chr,
  cls = cls,
  rows = NULL,
  r2_threshold = 0.001,
  max_distance_bp = 0,
  max_distance_variants = 1000,
  block_size = 1024,
  nthreads = 4
)
```

`Glist$sparseLD$prefix` is not a matrix object. It is the common prefix for the metadata, row-pointer, column-index, and value files consumed by `stblr_csr()`. `Glist$sparseLD` also records selected chromosomes, BED files, marker IDs, `cls`, allele frequencies, rows, distance limits, threshold, and block size.

Conceptually, CSR stores:

- `row_ptr`: offsets delimiting retained entries for each LD row;
- `col_idx`: column indices of retained marker pairs;
- `values`: retained marker correlations r.

The disk format stores the upper triangle with an implicit unit diagonal and zero-based column indices. Package functions handle this storage contract; analysis code should not manually alter index bases or binary files.

`r2_threshold` controls which correlations are retained by squared magnitude. The distance arguments can additionally restrict candidate marker pairs. A value of zero disables the corresponding distance restriction. Setting both distance limits to zero evaluates all marker pairs before thresholding and now requires `allow_full_ld = TRUE`. These settings change the LD approximation and should be recorded with analysis outputs.

6.1 Related Sparse-LD Helpers

The package source currently contains four related Rcpp wrappers:

Helper	Behavior	Current package status
`sparseLD_stream_CSR()`	Computes sparse LD and writes the disk-backed CSR files at `out_prefix`, returning a writing summary.	Exported and used by the public workflow.
`sparseLD_to_CSR()`	Computes the sparse LD representation and returns `row_ptr`, `col_idx`, and `values` in an in-memory R list.	Generated internal wrapper; not currently exported or separately documented.

Helper	Behavior	Current package status
`sparseLD_write_CSR()`	Currently delegates to `sparseLD_stream_CSR()` and writes the same disk-backed representation.	Generated internal wrapper; not currently exported or separately documented.
`sparseLD_read_CSR()`	Reads an existing disk-backed CSR prefix into an R list for inspection.	Exported.

Use `make_sparseLD()` for the high-level BED-to-disk workflow. It calls the exported `sparseLD_stream_CSR()` writer after resolving marker and sample metadata from `Glist`. The in-memory `sparseLD_to_CSR()` representation can be useful during development, but it materializes sparse arrays in R and is not the input contract used by `stblr_csr()`. The `sparseLD_write_CSR()` alias should not be treated as a separate stable public API while it remains unexported.

7 Inspecting and Validating CSR Files With `sparseLD_read_CSR()`

Before fitting, `sparseLD_read_CSR()` can inspect the files at `Glist$sparseLD$prefix`:

```
ld <- sparseLD_read_CSR(Glist$sparseLD$prefix, one_based = TRUE)

stopifnot(
  ld$nrow == length(cls),
  ld$ncol == length(cls),
  length(ld$row_ptr) == length(cls) + 1L
)
```

The reader returns the row pointers, column indices, values, dimensions, and metadata. `one_based = TRUE` makes returned column indices convenient for R inspection; `one_based = FALSE` exposes their zero-based disk convention. This option changes only the returned inspection object. The CSR samplers read the disk representation through package code and handle its index base.

Useful checks include confirming:

- matrix dimensions equal the selected marker count;
- row pointers have length $m + 1$;
- column indices fall within the expected base and dimensions;
- metadata records the intended `r2_threshold`, distance limits, and sample count;
- the number of stored values agrees with the final row pointer.

The binary-file and metadata details are documented further in Technical sparse LD and CSR.

8 Path B, Step 3: Fitting Summary-Statistics BLR With `stblr_csr()`

`stblr_csr()` combines the sufficient-statistics object with the disk-backed LD metadata in `Glist`:

```
fit_csr <- stblr_csr(
  stats = stats,
  Glist = Glist,
  method = "bayesC",
  pi_init = 0.001,
  h2 = 0.5,
  nit = 1000,
  nburn = 100,
  seed = 10,
  scheduled = FALSE
)
```

Its core data handoff is:

```
n + stats$yy
    -> phenotype variance and prior initialization

stats$wy + stats$ww + Glist$sparseLD$prefix
    -> marker-update and residual-score calculations
```

The wrapper infers the number of traits from `length(stats$yy)`. It takes `n` from the function argument or `stats$n`, and takes m from the function argument, `stats$m`, or `length(stats$ww[[1]])`. Explicitly supplying `n` and checking m before fitting makes the data contract easier to audit.

`stats$wy`, `stats$ww`, and `stats$yy` summarize the phenotype side and diagonal marker information. The files at `Glist$sparseLD$prefix` provide the retained off-diagonal marker relationships needed to update residual scores after marker effects change. The summary-statistics path therefore depends on both objects; neither is sufficient by itself.

9 Comparing BED-Direct and CSR Posterior Outputs

When both representations are prepared from the same individual-level data, `stblr_bed()` provides a useful comparison for `stblr_csr()`. Compare posterior quantities that share the same marker and trait order, such as posterior mean effects `fit$bm` and posterior inclusion probabilities `fit$dm`.

```
plot(fit_csr$bm[, 1], fit_bed$bm[, 1],
     xlab = "CSR posterior mean effect",
     ylab = "BED posterior mean effect")
```

The fits should agree **qualitatively** when markers, sample rows, phenotypes, scaling, allele frequencies, priors, and MCMC settings are matched. Strong signals and broad posterior patterns should be recognizable across paths. Exact equality is not expected because:

- CSR LD is a thresholded and possibly distance-limited approximation;
- BED and CSR samplers can use different update scheduling;
- finite MCMC chains introduce Monte Carlo variation;
- differences in initialization, residual adjustment, or prior settings change the posterior;
- floating-point and storage precision differ across calculations.

A comparison is an integration diagnostic, not a proof that either fit has converged or that the sparse-LD approximation is adequate.

10 Marker Order, Allele Frequency, Scaling, Rows, and LD Thresholds

The central practical invariant is:

```
Glist$rsids[[chr]][cls]
    == Glist$rsidsLD[[chr]]
    == rows of stats$wy and stats$ww
    == rows and columns of sparse LD
    == rows of any later annotation matrix
    == rows of fit$bm and fit$dm
```

The first equality should be tested directly:

```
stopifnot(
  !anyNA(cls),
  identical(Glist$rsids[[chr]][cls], Glist$rsidsLD[[chr]])
)
```

The remaining equalities are positional contracts. Sparse LD binary files do not supply biological marker IDs to repair a mismatch later. Retain `marker_id <- Glist$rsidsLD[[chr]]` beside the preparation outputs and attach or verify it whenever annotations and posterior summaries are joined.

Scaling also has to match. If `make_stats(..., scale = TRUE)` standardizes markers using `af`, then sparse LD must be constructed from the same selected samples and corresponding allele frequencies. If `rows` is used, use it in both calls and ensure `y` describes those rows. LD thresholds and distance limits should be treated as recorded model-input settings, not merely performance options.

The annotation workflow extends this contract with:

```
rownames(A) <- marker_id
stopifnot(nrow(A) == length(cls), identical(rownames(A), marker_id))
```

This is the handoff from the base CSR workflow to the models described in Annotation-informed models.

11 Validation Checklist

Before fitting `stblr_csr()`, verify:

1. `cls` has no missing values.
2. `Glist$rsids[[chr]][cls]` exactly matches `Glist$rsidsLD[[chr]]`.
3. `y` has `n` rows when `rows = NULL`, or describes the selected rows when `rows` is supplied.
4. `af` has one value per selected marker and follows `cls` order.
5. Every trait's `stats$wy` and `stats$ww` has length $m = \text{length(cls)}$.
6. `stats$yy` has one value per phenotype trait.
7. CSR `nrow` and `ncol` both equal m, and `row_ptr` has length $m + 1$.
8. CSR index bases are left to package functions; use `one_based` only when inspecting a returned R object.
9. CSR metadata records the expected sample count, threshold, and distance settings.
10. Annotation rows and marker/group vectors match `marker_id` exactly before annotation-informed models are fitted.
11. Posterior rows are interpreted in the same marker order.
12. Real analyses use adequate chains, convergence checks, and posterior predictive diagnostics.

12 Runtime, Disk, Reproducibility, and Convergence Cautions

Both sufficient-statistic construction and sparse-LD construction can be runtime-heavy. `block_size` and `nthreads` affect construction performance, while `r2_threshold` and distance restrictions also affect the statistical LD approximation. Record all of them.

Disk-backed CSR preparation writes multiple files sharing `Glist$sparseLD$prefix`. Treat them as one artifact: moving, replacing, or mixing files from different runs can invalidate the prefix. Use distinct prefixes for different marker sets, sample subsets, scaling choices, and LD thresholds.

For reproducibility, retain:

- marker IDs in model order;
- BED paths or immutable genotype-data identifiers;
- `cls`, `rows`, `af`, and phenotype preprocessing details;
- sufficient-statistic dimensions and trait names;
- the complete CSR prefix and its metadata;
- sampler priors, scheduling controls, seeds, and MCMC settings.

The short chains in workflow scripts demonstrate object handoffs. Real analyses require longer chains and appropriate convergence and posterior predictive checks. Compact output guidance is covered in Workflows and outputs.

13 Related Notes and Sources

- Data representations
- Technical sparse LD and CSR

- Technical summary statistics
- Workflows and outputs
- Annotation-informed models
- Complete sparse-LD/BED workflow
- Complete annotation workflow